Milestone 3: Performance Analysis and Optimization
Product Safety and Inspection Management System

Kelvin Kimani and Given Kangwa

June 5, 2026

1 Project Summary

The Product Safety and Inspection Management System is a PostgreSQL database developed to manage product categories, products, inspections, certifications, and recalls. The system supports product safety monitoring and regulatory compliance through efficient storage and retrieval of safety-related information.

2 Objective

The objective of this milestone is to evaluate database performance, identify slow queries, analyze query execution plans using EXPLAIN ANALYZE, optimize query performance through indexing, and demonstrate database automation using a trigger.

3 Methodology

3.1 Test Environment

The experiments were conducted using PostgreSQL 17.9 running on a 64-bit Windows operating system. The computer was equipped with an Intel Core i7 processor, 16 GB of RAM, and a 952 GB SSD for storage. The database used for testing contained approximately 1,000 rows of data.

3.2 Performance Evaluation Procedure

The evaluation procedure began by identifying slow queries and executing them using `EXPLAIN ANALYZE` to record the baseline execution plans. Next, database indexes were created on frequently used columns, and the same queries were re-run to capture post-optimization execution times. The results from before and after the optimization were then compared and interpreted. Finally, a database trigger was implemented and tested.

4 Query Performance Analysis

4.1 Query 1: Products and Inspections Join

Purpose

This query joins product information with inspection information using the `product_id` column.

Execution Plan Before Indexing

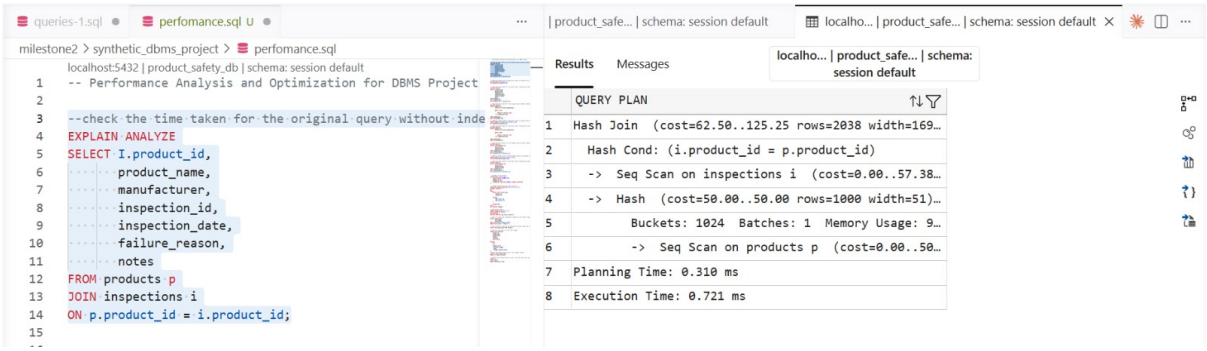

Figure 1: Query 1 Execution Plan featuring Sequential Scans and Hash Join prior to Indexing

Execution Plan After Indexing

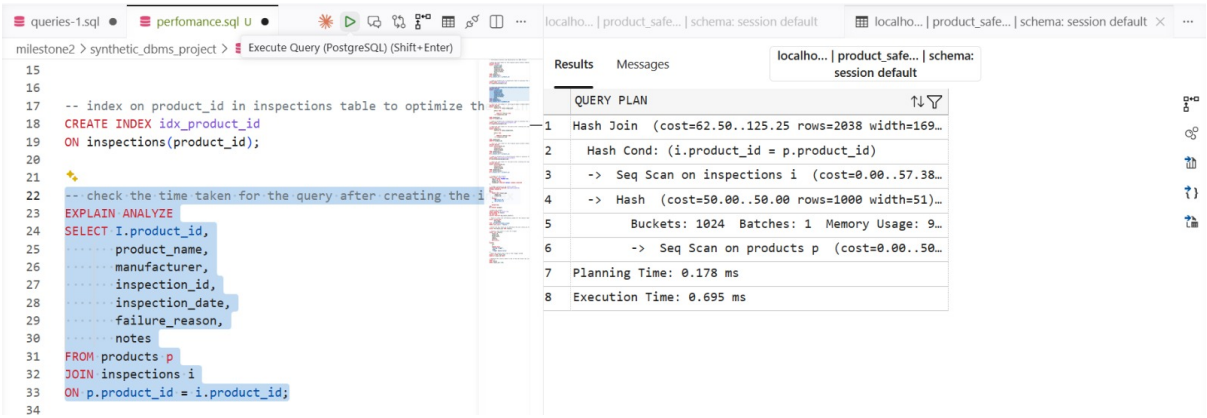

Figure 2: Query 1 Execution Plan showing performance changes following the creation of `idx_product_id`

Results

Execution time before indexing: 0.721 ms

Execution time after indexing: 0.695 ms

Performance improvement: 3.61%

Interpretation

PostgreSQL utilized a Hash Join operation with sequential scans on both tables. Due to the small dataset size (approximately 1000 rows), the query planner favored scanning the full tables sequentially since loading them entirely into memory is efficient. However, a marginal improvement of 3.61% was still captured.

4.2 Query 2: Product Inspection Ranking

Purpose

This query ranks products according to the number of inspections performed.

Execution Plan Before Indexing

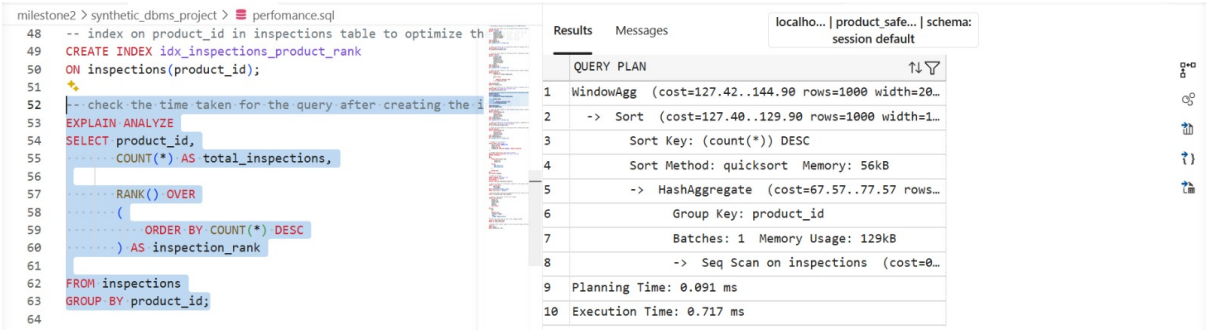

Figure 3: Query 2 Execution Plan showing Window Aggregation and Quicksort processing before Indexing

Execution Plan After Indexing

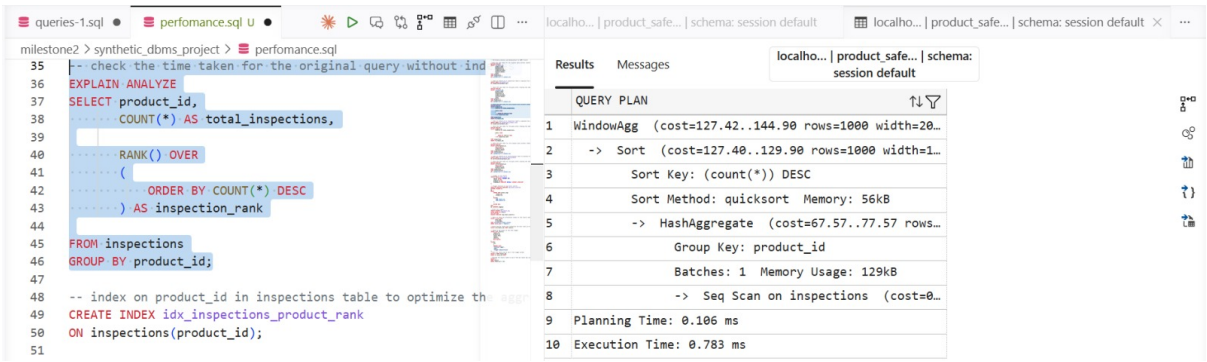

Figure 4: Query 2 Execution Plan maintaining a Sequential Scan strategy after creating `idx_inspections_product_rank`

Results

Execution time before indexing: 0.717 ms

Execution time after indexing: 0.783 ms

Performance improvement: No significant improvement

Interpretation

The query required deep aggregation, grouping, sorting, and window ranking functions (`RANK() OVER`). PostgreSQL bypassed the index and performed a full sequential scan because computing global aggregates across the entire scope requires processing almost every row in the dataset regardless.

4.3 Query 3: Inspections and Certifications Join

Purpose

This query joins inspection information with certification information using the `product_id` column.

Execution Plan Before Indexing

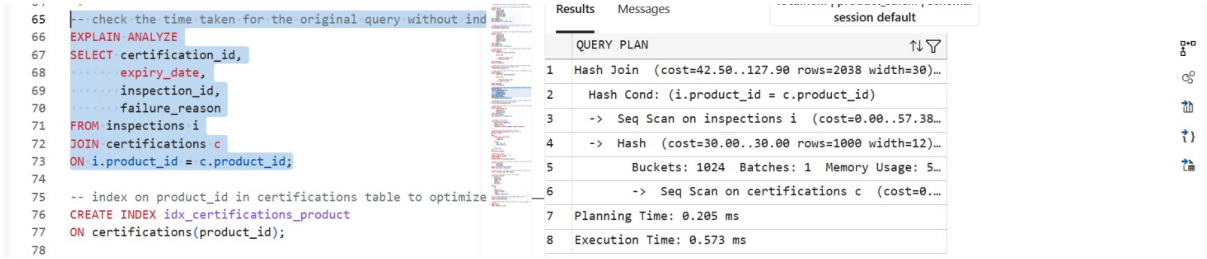

Figure 5: Query 3 Execution Plan showing base Hash Join and Sequential Scans before indexing

Execution Plan After Indexing

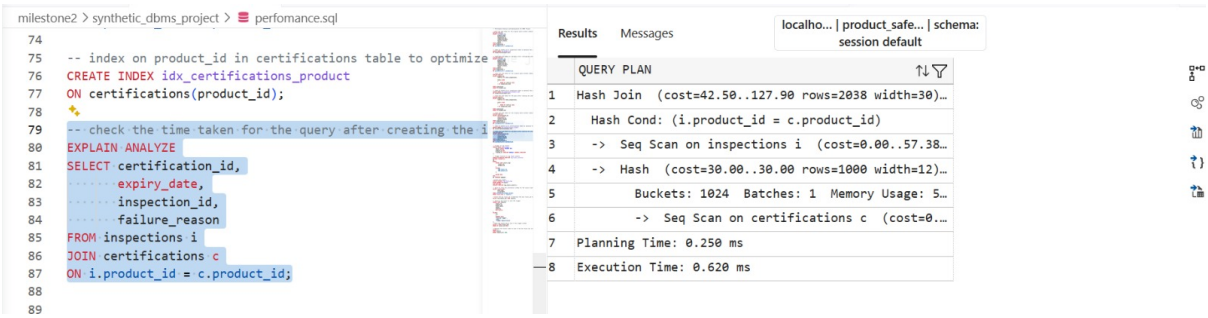

Figure 6: Query 3 Execution Plan showing unchanged node logic after creating `idx_certifications_product`

Results

Execution time before indexing: 0.573 ms

Execution time after indexing: 0.620 ms

Performance improvement: No significant improvement

Interpretation

PostgreSQL evaluated the cost metrics and maintained an identical structure using Sequential Scans and a Hash Join strategy. For low volume tables, building a hash table in cache remains significantly faster than handling index disk pointer leaf lookups.

5 Index Design and Justification

5.1 Indexes Created

To improve query performance, three indexes were created: `idx_product_id` on the `inspections(product_id)` column, `idx_inspections_product_rank` on the `inspections(product_id)` column, and `idx_certifications` on the `certifications(product_id)` column.

5.2 Justification

These indexes were created because the `product_id` column acts as a primary vector in relational mapping and `JOIN` operations. While the current minor testing dataset bounds allow sequential scanning to stay extremely competitive, these structural indexes establish scalability checkpoints that prevent severe linear execution path decay ($\mathcal{O}(N)$) as the live environment grows.

6 Performance Comparison

Query	Before (ms)	After (ms)	Improvement
Products and Inspections Join	0.721	0.695	3.61%
Product Inspection Ranking	0.717	0.783	No Improvement
Inspections and Certifications Join	0.573	0.620	No Improvement

Table 1: Performance Comparison Before and After Indexing

7 Trigger Implementation

7.1 Purpose

A trigger was implemented to automatically record recall information whenever a new recall is inserted into the recalls table.

7.2 Trigger Creation

```
-- Trigger function to log recall inserts
CREATE OR REPLACE FUNCTION log_recall_insert()
RETURNS TRIGGER AS
$$
BEGIN
    INSERT INTO recall_log(
        recall_id,
        product_id
    )
    VALUES(
        NEW.recall_id,
        NEW.product_id
    );

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

--Create the trigger.
CREATE TRIGGER trg_recall_log
AFTER INSERT ON recalls
FOR EACH ROW
EXECUTE FUNCTION log_recall_insert();
```

Figure 7: PL/pgSQL Trigger Function implementation `log_recall_insert()` and bound execution hook `trg_recall_log`

7.3 Trigger Testing

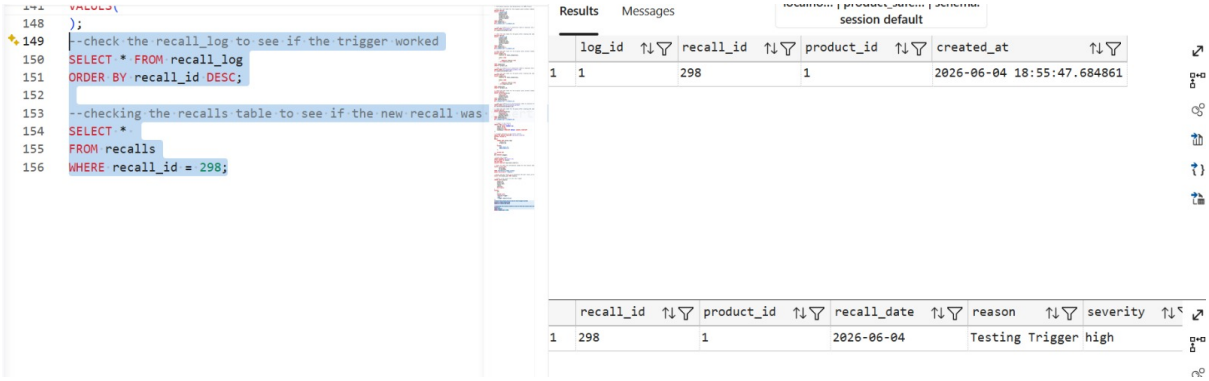

Figure 8: Verification check showcasing real-time insertion mapping directly onto the transactional log target `recall_log`

7.4 Interpretation

Whenever a new recall record is written into the `recalls` table, the event hook automatically routes contextual data identifiers downstream into the `recall_log` schema block. This verifies transactional integrity and isolates an independent audit footprint.

8 Conclusion

Three database patterns were safely monitored using `EXPLAIN ANALYZE` benchmarks across optimization boundaries. The *Products and Inspections Join* realized an indexing runtime change of 3.61%. Both the sorting constraints in the *Product Inspection Ranking* query and the tiny footprint of the *Inspections and Certifications* query led the PostgreSQL query planner to favor sequence scans for peak execution cost efficiency. Finally, an audit logging automation architecture was completely validated using trigger mechanisms.